

STAT 578: Mini-Project I

Contrastive and Non-contrastive Self-Supervised Learning

Fall 2026 • Week 4

Overview

You will **reproduce**, **probe**, and **interpret** two families of self-supervised methods on CIFAR-10: one *contrastive* (SimCLR) and one *non-contrastive* (BYOL or SimSiam). The goal is not to chase state-of-the-art accuracy, but to *measure* what these methods do to a representation — alignment, uniformity, and the two faces of collapse — and connect what you see to the theory from Weeks 2–3.

Learning goals. By the end you should be able to (i) implement and train a contrastive and a non-contrastive objective; (ii) measure alignment, uniformity, effective rank, and normalized-output std, and read them correctly; (iii) demonstrate collapse and its prevention through controlled ablations; and (iv) explain your observations using the statistical ideas from lecture.

Logistics

- **Groups** of two (self-organized; groups of one are allowed with instructor approval). Please let instructor know your group by Friday, September 15, 11:59 pm CST.
- **Compute:** CIFAR-10 from scratch on a single modest GPU. A full run is ~ 100 epochs and takes roughly **2–4 hours depending on the GPU** (faster on an A100, slower on a shared RTX 2080/3060); use fewer epochs while debugging. No pretrained weights.
- **Submit:** a 4–6 page report (PDF), your code, and the required figures. Each group member should contribute to both experimentation and interpretation. The final report should briefly describe individual contributions.
- **Starter code** is provided (see below): data, models, the three methods with ablation flags, the diagnostics, a training loop, and a linear-probe script. You should read and modify it, not treat it as a black box.

The starter code

file	contents
<code>data.py</code>	CIFAR-10 with a two-view SSL augmentation
<code>models.py</code>	CIFAR ResNet-18 encoder (h , 512-d), projector g , predictor q
<code>methods.py</code>	SimCLR, SimSiam, BYOL; ablations <code>--no_predictor</code> , <code>--no_stopgrad</code>
<code>metrics.py</code>	<code>alignment</code> , <code>uniformity</code> , <code>output_std</code> , <code>effective_rank</code>
<code>train.py</code>	pretraining loop; logs <code>metrics.csv</code> per epoch; saves <code>encoder.pt</code>
<code>linear_probe.py</code>	freeze encoder, train a linear classifier, report test accuracy
<code>diagnostics.py</code>	<code>alignment/uniformity/std/effective-rank</code> + singular-value spectrum plot

See `README.md` for exact commands.

Part 1 — Reproduce

1. Train **SimCLR** and **one** non-contrastive method (BYOL *or* SimSiam) on CIFAR-10 with the provided code (a reasonable budget: batch size 256, ~ 100 epochs).
2. For each, run the **linear probe** and report test accuracy.
3. Sanity-check that a healthy run has `output_std` $\approx 1/\sqrt{d}$ and a large effective rank; report both.

Part 2 — Probe (the core of the project)

Design and run experiments; *measure*, do not just train.

(a) Collapse without asymmetry. Retrain with the predictor removed (`--no_predictor`), and—using **SimSiam**—with the stop-gradient removed (`--no_stopgrad`). Track `output_std` and `effective_rank` over training. *Why SimSiam for the stop-gradient ablation?* BYOL’s target is a separate EMA network whose parameters carry `requires_grad=False`, so “removing” its stop-gradient would just discard gradients rather than truly ablate it; SimSiam’s explicit `.detach()` is the clean place to turn stop-gradient off. (The starter code will remind you of this.)

- Which ablation collapses? Which *kind* of collapse does each diagnostic reveal — *complete* (`output_std` $\rightarrow 0$) or *dimensional* (small `effective_rank`)?
- Include the singular-value spectrum (from `diagnostics.py`) for a healthy vs. a collapsed encoder.

(b) Contrastive knobs. For SimCLR, sweep the temperature $\tau \in \{0.1, 0.2, 0.5, 1.0\}$ (and, if time allows, batch size $\in \{128, 256, 512\}$). For each, report **alignment**, **uniformity**, and **linear-probe accuracy**.

Required diagnostics. Every run should report, over training and/or at the end: alignment, uniformity, `output_std`, effective rank, and (for evaluated encoders) linear-probe accuracy.

Part 3 — Connect

Interpret your results using lecture material. At minimum, address:

1. **Why negatives matter.** SimCLR without negatives is pure alignment. Explain, using the collapse argument (Weeks 2–3), why that collapses, and what the negatives (uniformity term) add. Relate the τ sweep to the alignment–uniformity decomposition.
2. **Existence vs. attractiveness.** For the non-contrastive method, the constant solution *remains* a valid optimum. Explain why stop-gradient / the predictor change whether training *moves toward* it — and connect that to which of your ablations collapsed.
3. **Two collapses.** Explain why `output_std` and `effective_rank` can disagree, and which detects which failure.

Deliverables

- **Report (4–6 pages)**, workshop-paper style: setup; reproduction results; probing experiments and figures; connection to theory; brief conclusion / limitations.
- **Code** (your modified starter code) with a one-command way to reproduce a key figure.
- **Required figures:** (1) training curves (loss, alignment, uniformity, `output_std`, effective rank) for both methods; (2) a collapse-ablation curve showing `output_std` (and/or effective rank) with vs. without stop-gradient; (3) a healthy-vs-collapsed singular-value spectrum; (4) SimCLR alignment/uniformity/accuracy vs. τ ; (5) a linear-probe accuracy table.

Grading rubric

Component	Weight
Reproduction & correctness (methods train; probe/metrics implemented correctly)	25%
Depth & correctness of the probing analysis (ablations, sweeps, diagnostics read right)	35%
Connection to theory (collapse, alignment/uniformity, existence vs. attractiveness)	25%
Clarity of writing and figures	15%

Weighting the analysis and theory above the reproduction is deliberate: this is a course on *statistical foundations*, not a leaderboard.

Tips & policies

- Start early — a training run takes hours. Debug with a few epochs first.
- Defaults in the starter code are a starting point, not tuned optima; exploring is part of the task.
- **AI policy:** you may use generative AI for debugging and writing clarity, but the experimental design, code, results, and interpretation must be your own understanding, and any AI assistance must be acknowledged (see the syllabus).